

RIPPOTAI ERP (INOS)

Frontend-to-Backend Integration

How the React application is wired to the backend: request path, authentication, state, routing, endpoints, and integration defects

Item	Detail
Document	Frontend integration design and endpoint reconciliation
Version	1.0
Date	10 October 2026
Scope	rippotai-erp-frontend (React application) and its calls into the backend service
Basis	Frontend source, package.json, vite.config.js, the backend route inventory, and a production build of the frontend
Classification	Internal

Contents

- 1 Executive summary
- 2 Scope and method
- 3 Frontend architecture
- 4 Configuration and build
- 5 Request path: two HTTP clients
- 6 Authentication and session wiring
- 7 Client state, caching and refresh
- 8 Route access and role gating
- 9 Feature-to-backend integration map
- 10 Endpoint reconciliation results
- 11 Real time and notifications
- 12 Files and documents
- 13 Zoho owner-key wiring
- 14 Frontend security observations
- 15 Quality and maintainability
- 16 Recommendations
- 17 Limitations
- Appendix A Endpoint mismatch register
- Appendix B Reproduction

1 Executive summary

The frontend is a React 19 single-page application built with Vite. It talks to the backend over HTTPS with a Bearer token, through a single Redux Toolkit Query (RTK Query) API. Almost every feature is wired in the same way: a feature module injects endpoints into one shared API, mutations invalidate cache tags, and lists refresh by polling. The design is consistent and easy to extend.

Measured against the backend's actual routes, the integration is largely sound but has defects that matter. Of 795 endpoint definitions in the live application, 743 (93.5%) match a backend route with the same method and path. The other 52 do not: 39 call paths the backend does not define, 6 call the right path with the wrong HTTP method, and 7 build URLs too dynamically to verify statically. Those 52 calls will fail at runtime when the affected screens are used.

Headline results

Measure	Result	Why it matters
Endpoint definitions analysed (live modules)	795	Five API modules are never imported and were excluded as dead code.
Matched to a backend route (method and path)	743 (93.5%)	The main integration works.
Path absent from backend	39	Screens calling these paths will get 404 responses. Concentrated in procurement, site operations and inventory.
Right path, wrong method	6	Calls such as POST to exports that the backend serves as GET.
Dynamic URL, not verifiable statically	7	Needs a runtime check.
Working calls landing on unguarded backend routes	At least 469 of 743 (63%)	The frontend's use of tokens is not enforced by these routes. Same root cause as backend finding SEC-01.
Endpoint names defined in more than one API file	38	Only four injections set overrideExisting; which definition is active depends on module evaluation order.
Real-time gateway used by the frontend	No	No Socket.IO client is installed. Notifications are polled every 15 seconds.
Production build	Passes	Main JavaScript bundle 4.27 MB (1.16 MB gzipped); no route-level code splitting.

Priorities

1. Fix or remove the 52 mismatched or unverifiable calls, starting with procurement and site operations, where most of the 39 absent paths sit.

2. Correct the role and paywall checks in the frontend. Both depend on fields the backend does not return, so they currently evaluate to false for every user.
3. Remove the CDN secret from the browser build path and the Zoho owner-key derivation from localStorage. Neither should be trusted by the backend.
4. Decide on real-time delivery. Either adopt the existing Socket.IO gateway in the frontend or remove the unused server capability and document polling as the design.
5. Consolidate the 38 duplicated endpoint definitions so each operation has one owner.

Scope of the result. The reconciliation matches frontend source literals against backend controller decorators. It proves that a route with the same method and path exists. It does not prove the handler behaves correctly, that the request body matches the DTO, or that the response shape is what the screen expects.

2 Scope and method

This document covers the frontend's wiring into the backend: how requests are built and sent, how sessions are kept, how server state is cached and refreshed, how routes are guarded, and which backend endpoints each feature calls. It does not cover visual design, component behaviour, or accessibility.

2.1 Method

1. Structure review of the React source tree, configuration, and dependency manifest.
2. Reading of the two HTTP clients, the RTK Query base API, the store, the auth context, routing helpers, and the notification client.
3. Extraction of every endpoint definition by parsing each `builder.query` and `builder.mutation` block, so that each URL is paired with its own HTTP method.
4. Normalisation of URLs (template expressions and route parameters are treated as wildcards) and matching against the backend route inventory built from controller decorators.
5. Classification of unmatched calls into absent path, method mismatch, and dynamic, and removal of modules that are never imported.
6. Installation of frontend dependencies with a clean install and a production build with vite build.

2.2 What counts as a match

A call matches when a backend route exists with the same HTTP method and a path with the same number of segments, where each literal segment is equal and each parameter (including composed template expressions) matches any value. A matched call is not proof of correct behaviour. The guard status reported in Section 10 is taken from the controller file, so it is conservative: a file that imports a guard is counted as guarded.

3 Frontend architecture

Layer	Technology and role
UI	React 19 with react-router-dom 7 for routing. Radix UI primitives and Tailwind CSS for components. recharts for charts. lucide-react for icons.
Forms	react-hook-form with zod through @hookform/resolvers.
Server state	Redux Toolkit 2 with RTK Query, through react-redux 9. One shared API instance (baseApi).
Local state and context	React context for authentication (AuthContext) and chat (CliqChatContext). Component state elsewhere.
HTTP	An axios instance (src/lib/api.js) for 17 modules and the RTK fetchBaseQuery for the 58 modules that inject endpoints. Raw fetch in a few places.
Export and documents	exceljs, xlsx, jspdf, html2pdf.js and html2canvas for spreadsheets and printable documents.
Drag, layout and grids	dnd-kit for sorting, react-grid-layout for dashboard widgets, react-resizable-panels for split views.
PWA	vite-plugin-pwa with auto-update and a precache of 14 entries.
Testing	11 test files. No test script is defined in package.json, so no npm command runs them.

3.1 Project structure

Folder	Contents	Count
src/pages	Screens, grouped by domain (projects, materials, boq, site-ops, leads, settings and others)	214 files
src/components	Shared and domain components (boqs, documents, site-ops, leads, command-center and others)	175 files
src/api	RTK Query endpoint modules, grouped by backend domain	62 files
src/router	Route definitions in 19 route files, plus the route generator and protection wrappers	About 200 route entries
src/store	The store, the base API with 157 declared cache tags, and the shared document-cache middleware	3 files
src/lib	API base URL, axios instance, notification mapping, route helpers, rich-text cleaning, mock-like helpers	17 files
src/context	AuthContext: session, login, signup, logout, profile update	1 file
src/hooks, src/widgets	Shared hooks, and dashboard widgets (16 files in widgets)	Several files

4 Configuration and build

4.1 Backend address

`src/lib/config.js` selects the backend origin. It uses the `VITE_BACKEND_URL` build variable when set. Otherwise, production builds fall back to `https://erp-api.rippotaiarchitecture.com` and development builds to `http://localhost:5000`. Any trailing slash or `/api` suffix is removed, and the result is exposed as `API_BASE_URL` (origin plus `/api`) and `API_URL` (adds `/v1`). This is a sound, single point of configuration. The value in the repository's `.env` file is not reproduced in this document.

4.2 Build behaviour

- A production build (vite build) completes without errors in the verification run.
- The main JavaScript bundle is 4.27 MB (1.16 MB gzipped), with a further 933 kB chunk for the Excel checklist export. No route is lazily loaded: the repository contains no `React.lazy` usage, and `vite.config.js` raises the chunk-size warning limit to 5,000 kB rather than splitting the code.
- The build reports an ineffective dynamic import: `src/lib/config.js` is imported both statically and dynamically (by `mockApi.js`), so the dynamic import does not create a separate chunk.
- The service worker precaches 14 entries totalling about 5.6 MB. Users on slow connections download most of the application on first visit.
- `vercel.json` rewrites all paths to `index.html`, which is required for client-side routing on static hosting.

Consequence. Because the entire application is one bundle, first-load cost grows with every feature. Route-level code splitting is the single largest performance improvement available without changing behaviour.

5 Request path: two HTTP clients

The application has two HTTP clients. Both attach the same Bearer token, but they differ in how they handle errors and side effects, so the choice of client determines behaviour.

Client	Where it is used	Behaviour
RTK Query (<code>fetchBaseQuery</code>), via <code>baseApi</code>	58 API modules that inject endpoints. This is the primary path for almost all data.	Adds Authorization from <code>bc_token</code> . Also adds <code>x-cdn-secret</code> when <code>VITE_CDN_TOKEN</code> is set at build time (Section 14). Sends credentials: include, although the backend issues no cookies.
axios instance (<code>src/lib/api.js</code>)	17 modules, mostly older screens and services.	Adds Authorization. On 401, clears the session and redirects to <code>/login</code> . On 402, raises an upgrade event. On any non-GET response, dispatches <code>bc:dashboard-refresh</code> for listeners in four files.
Raw fetch	ZohoCrmTester page, the mockApi settings helper, the Cliq attachment preview.	Bypasses both clients. Each call sets headers by hand, so it does not get the 401 handling.

5.1 Observations

- **The 401 path exists only in the axios client.** RTK Query calls do not redirect on 401. A request that fails with an expired token shows a screen error rather than returning to login. The session is cleared only when `AuthContext`'s refresh step fails on the next load.
- **The 402 path is dead.** The axios client and the plan-guard helpers handle HTTP 402 as an upgrade signal. The backend never returns 402 (searched across all TypeScript sources), so the upgrade prompt cannot be triggered by the server.
- **The credentials setting is misleading.** The base query comment describes cookie-based auth, but authentication is entirely header-based. `credentials: include` is harmless here but suggests a session mechanism that does not exist.
- **The refresh event is a cross-cutting bus.** Every successful mutation made through axios broadcasts `bc:dashboard-refresh`. RTK Query mutations do not dispatch it, so dashboards updated through RTK mutations depend on tag invalidation instead.

6 Authentication and session wiring

The session model is simple. The backend issues a signed token and the user object. The frontend keeps both in browser storage and sends the token on every request.

6.1 Sequence

1. The login form calls POST /auth/login through the RTK mutation. The response contains a token and a user.
2. AuthContext stores the token under bc_token and the user under bc_user in localStorage, and sets the user in React state.
3. On every later API call, prepareHeaders adds Authorization: Bearer with the stored token.
4. On application start, AuthContext reads bc_user for an immediate render, then calls GET /auth/me to refresh it. If that call fails, the session is cleared.
5. Logout calls POST /auth/logout, then clears both keys regardless of the response.
6. Profile updates merge the server response into bc_user, so the stored user always reflects the last update.

6.2 What this design relies on

Aspect	Current behaviour	Assessment
Token storage	localStorage keys bc_token and bc_user; localStorage is referenced 39 times across the application	Readable by any script running in the page. A single cross-site scripting flaw exposes the session. Consider httpOnly cookies, which the backend would then need to support.
Session refresh on load	/auth/me validates the token and refreshes the user	Sound. The backend re-checks the session row on each request.
Logout	Server call, then local clear regardless of result	Sound. Local state is always cleared.
Client-side user trust	UI decisions read role and plan from bc_user	The server is authoritative for API calls, but the UI trusts stored copies. See Section 8.
Signup	Signup stores a session immediately	Matches the backend's behaviour: no email verification step before a session is issued (backend finding SEC-02).

7 Client state, caching and refresh

7.1 Single API with tagged cache

Every endpoint declares which tags it provides or invalidates. Lists provide a tag and mutations invalidate it, so the cache refreshes without manual calls. The base API declares 157 tags. A dedicated middleware (shared-document-cache.js) invalidates 14 distinct tags after any mutation whose name matches a pattern for projects, briefs, recces, scope, budgets, planners, quotations, procurement, BOQs and purchase or work orders. This is deliberate: a document save can change other documents on the backend, so the middleware invalidates broadly.

7.2 Polling

Real-time behaviour is approximated by polling. The table below lists the intervals found in the source.

Screen or widget	Mechanism	Interval
Notifications bell	pollingInterval, plus refetch on focus and reconnect	15 seconds
Cliq mini widget and context	pollingInterval, faster when a conversation is open	15 seconds open, 60 seconds closed, 4 seconds in messages
Tasks page	pollingInterval	30 seconds
Calendar view	pollingInterval, plus refetch on focus and reconnect	30 seconds
Reminder sync panel	pollingInterval	30 seconds
Command Center	setInterval invalidation of refresh tags while the tab is visible, plus a manual rescan	30 seconds
Inventory widget	pollingInterval	60 seconds

Polling is simple and robust across restarts, but it multiplies backend load with the number of open tabs. Each Command Center tab, for example, invalidates its aggregate every 30 seconds. The backend's aggregate is fresh for 15 seconds and held in Redis for 120 seconds, so polling is largely absorbed by the cache, but the cost should be measured.

7.3 Notification preferences

Notification group preferences are stored only in localStorage (use-notification-preferences.js) and are not sent to the backend. Preferences therefore do not follow the user across browsers and are not visible to the server, which means server-side notification generation cannot respect them.

8 Route access and role gating

Route protection has three layers in the frontend: a route type, a protection wrapper, and in some pages a role or plan check. Only the first two are wired in practice.

Mechanism	Implementation	Status
Route types	public (logged-out only), noauth (open), raw, layout (protected). Defined in generate-routes.jsx.	Wired.
Authenticated wrapper	Protected renders a loading state while AuthContext is ready, redirects to /login without a user, and redirects blocked roles to /dashboard.	Wired, but only checks that a user exists on the client.
blockRoles	Five protected layouts block the role "client" (projects, site operations, design studio and the admin console).	Likely inert. No backend code assigns a role named client; the backend uses uppercase CLIENT in one DTO list and never assigns it as a role. The check compares case-sensitively, so it probably never matches.
Role comparisons in pages	Pages compare user.role against ADMIN, admin, SUPERADMIN and USER in a few places.	Inconsistent casing. Authorisation is not enforced by these checks; the backend is the only real enforcement point, and most of its routes are unguarded (backend finding SEC-01).
Plan gating	uselsPaid reads user.plan and user.is_super_admin. PaidOnly and SuperAdminOnly wrap features.	Inert. The backend's login and profile responses do not include plan or is_super_admin, so both checks return false for every user. PaidOnly is not used in any JSX.
Upgrade prompt	The 402 handler raises inos:upgrade-required.	Dead. The backend never returns 402.
Debug console	ZohoCrmTestConsole is registered as a route in crm.routes.jsx.	Exposed. A developer tool is reachable by URL from the CRM routes; check whether it sits behind the protected layout.

Implication. The frontend's role and plan checks look like access control but are not. Any screen hidden by a check is still reachable by URL, and the API it calls is authorised only by authentication, or not at all on unguarded routes. The checks should be replaced by server-issued permissions, returned with the user and enforced by the backend.

9 Feature-to-backend integration map

The table shows each frontend API module group, the backend domain it calls, and the reconciliation result for its endpoints in the live application.

Frontend group	Backend domain	Total	Match	Absent	Method	Dyn.
procurement (procurement, inventory, delivery challans, site ops)	material-procurement, inventory, delivery-challans, site-ops	165	140	25	0	0
documents (documents, drawings, briefs, payments, planners)	documents, drawings, brief, payment-schedules, project-planner	145	142	2	1	0
site-ops (visits, DPR, RFIs, snags, QC)	site-ops, architect, dpr	71	57	7	3	4
users (users, roles, teams, signatures)	users, rbac, role-apps, team	61	61	0	0	0
meta (units, terms, search, settings, CDN)	units, terms-templates, search, settings, cdn	61	61	0	0	0
projects (projects, clients, types, command center)	projects, command-center, dashboards	56	55	1	0	0
boq (BOQs, templates, library, catalogue)	boqs, boq templates, library	56	54	1	1	0
connectors (tasks, calendar, leads, Zoho, Cliq, OneDrive)	google, microsoft, zoho, leads, sync	53	53	0	0	0
vendors (vendors, shortlists, categories)	vendors, vendor-categories, shortlists	44	41	3	0	0
auth (sessions, tokens, OAuth, verification)	auth	26	23	0	0	3
workspace (notes, tasks, calendar feed)	workspace, tasks	25	25	0	0	0
automation	automation	17	17	0	0	0
reports	reports	9	9	0	0	0
engagement (notifications, activity logs)	engagement	5	5	0	0	0
Other	root	1	0	0	1	0
Total (live modules)		795	743	39	6	7

Table 9.1. Endpoint reconciliation by frontend module group. Five API modules that are never imported are excluded.

Three areas carry most of the defects: procurement (25 of the 39 absent paths, mostly inventory, delivery challans and site inventory), site operations (visits and QC), and vendors (shortlists). Those areas are

where the frontend and backend were evolved independently, and where the naming in the frontend no longer matches the backend's controllers.

10 Endpoint reconciliation results

Result	Count	Interpretation
Matched (method and path)	743	Route exists. Behaviour not verified.
Absent path	39	Route does not exist. Calls will return 404 at runtime.
Method mismatch	6	Route exists under another method. Calls will return 404 or 405.
Dynamic, unverified	7	URL is composed at runtime. Needs a request-level test.
Dead modules excluded	60 definitions in 5 files	Modules never imported. Their endpoints cannot run, but they should be deleted.

10.1 Guard exposure of working calls

Of the 743 matched calls, at least 469 (63%) reach backend routes whose controller has no framework guard. The highest counts are in documents (128), procurement (105), meta (54), users (50) and BOQs (35). The frontend sends a valid token on these calls, but the backend does not check it on those routes. That is the same root cause as backend finding SEC-01, and the reconciliation shows how much of the live application depends on it.

10.2 Duplicated definitions

38 endpoint names are defined in more than one API file. Examples include `getTeamMembers` and `addTeamMember` (`team.api.js` and `team-members.api.js`), `raiseRfi` and `respondToRfi` (two definitions in `site-ops.api.js`), and `createDailySiteReport` (three definitions across `site-ops` and `daily-site-report`). RTK Query keeps the first definition registered for a name unless `overrideExisting` is set, and only four injections set it. The active definition, and therefore the URL a screen calls, depends on module evaluation order. Several duplicates differ in path or method, so a change to one file can silently change which request is sent.

10.3 Dead code

Five API modules are never imported anywhere: `team-members.api.js`, `site-visits.api.js`, `daily-site-report.api.js`, `checklist.api.js` and `zoho-crm.api.js`. Their endpoints cannot run. The checklist module holds four of the broken QC paths, so its presence makes the codebase look more broken than it is at runtime, and it makes review harder.

11 Real time and notifications

The backend exposes a Socket.IO namespace (/notifications) that authenticates connections with the session token and joins each user to a private room. It is designed to push notifications to clients as they are created, with a Redis adapter for multiple instances.

The frontend does not use it. package.json includes no Socket.IO client, and no source file opens a socket connection to the notifications namespace. Notifications are fetched through RTK Query: the bell polls every 15 seconds and refetches on focus and reconnect. Lists elsewhere refresh on their own intervals.

Decision needed. The server capability exists and would remove most polling load. Either adopt it in the frontend, which requires a client library, token handling on connect, and reconnection logic, or remove it from the backend and document polling as the design. Leaving both in place means the team pays the complexity cost of the server path without receiving its benefit.

11.1 Notification routing

The frontend maps each notification's entity type to a destination screen (lib/notifications.js). Destinations cover purchase orders, work orders, challans, BOQs, planners, RFIs, mock-ups, visits, snags, daily reports, QC, team, automation rules and document types. A notification whose entity type has no mapping falls back to the 'other' group with no destination. Deleted entities return no destination, which is correct.

12 Files and documents

- **Document files** are uploaded to POST /documents/:id/file, versions to /documents/:id/versions, and attachments to /documents/:id/attachments. These are multipart requests built with FormData in the document API module.
- **Signatures, avatars and OneDrive files** also send FormData through their own API modules.
- **The CDN upload hook** (useUploadFileToCdnMutation, calling POST /cdn/upload) is defined but has no callers in the application. Uploads for live screens therefore go through the document and signature routes instead.
- **Downloads** use backend download routes such as GET /documents/:id/download. Export endpoints for BOQs, snags, DPR and planners are GET in the backend but are called as POST by the frontend in some places (Section 10 and Appendix A).

The backend accepts uploads up to 500 MB on the CDN route, buffered in memory (see backend finding SEC-12). Document routes apply their own limits, which were not reviewed in this pass. The frontend does not enforce a size limit before uploading, so a large file travels the full distance before the backend rejects it.

13 Zoho owner-key wiring

Zoho integrations are addressed by an owner key that appears in the URL path. The frontend builds that key from the signed-in user's identifier, read from `bc_user` in `localStorage`. The calendar and Zoho CRM API modules define a `getOwnerKey` helper that does this, and the task and Cliq modules reference the owner key in the same way. The owner key is then sent as a path segment, for example `/zoho/bigin/{ownerKey}/modules/Deals`.

Defect. The identity used to select a Zoho account comes from browser storage and is sent as a path parameter. The backend's Zoho controllers do not check the caller's session, so any caller who knows a user identifier can address that user's Zoho account (backend finding SEC-06). The frontend cannot fix this alone. The backend should resolve the account from the authenticated user, and the frontend should stop sending an owner key.

14 Frontend security observations

Observation	Evidence	Severity
Session token readable by page scripts	bc_token in localStorage; localStorage is referenced 39 times in total	High
CDN secret would ship in the bundle if set at build time	baseApi adds x-cdn-secret from VITE_CDN_TOKEN to every request. The repository's .env does not set it, and production build settings are not visible. The only caller is an unused hook.	High if set; currently dormant
Authorisation decided in the browser	Route blocks, role comparisons and plan checks (Section 8)	High
Zoho account selected by client-supplied identifier	Section 13	High
Debug console registered in application routes	ZohoCrmTestConsole route in crm.routes.jsx	Medium
Hand-written HTML sanitisers	cleanRichText (rich-text.js) and termHtml (WorkOrderDocument.jsx) remove scripts and some attributes before dangerouslySetInnerHTML. The bundle includes a DOMPurify module, but these two paths do not use it.	Medium
Raw fetch bypasses shared client	ZohoCrmTester, mockApi settings helper, Cliq attachment preview	Low
Notification preferences stored client-side only	use-notification-preferences.js	Low

Output encoding is handled reasonably in most places. Plain text is escaped, and the two HTML renderers apply filters. The recommendation is to replace the hand-written filters with one vetted sanitiser, used at every point where HTML is rendered, so the policy lives in one place.

15 Quality and maintainability

Indicator	Finding	Note
Build	Passes	Single bundle; warning about an ineffective dynamic import.
Tests	11 test files; no test script in package.json	Tests are not run by any npm script. Not executed in this review.
Lint	A lint script exists (eslint .)	Not executed in this review.
Duplicate endpoint names	38	Section 10.2
Dead API modules	5	Section 10.3
Misnamed module	mockApi.js performs real persistence through the settings API	The name misleads readers into thinking it is a stub. Rename it.
Mixed HTTP clients	Two clients plus raw fetch	Consolidate on one client with consistent error handling.
Route code splitting	None	Section 4.2
Role and plan logic	Duplicated across pages, with inconsistent casing	Centralise in one permissions hook fed by the server.

16 Recommendations

Immediate (this sprint)

- Replace or remove the 39 absent-path calls and 6 method mismatches. Use the register in Appendix A. Where a backend equivalent exists (for example, /site-ops/qc to site-ops/checklists, /vendor-shortlists to project-shortlists, /inventory/receive to inventory/receive-delivery), update the frontend. Where none exists, decide whether the backend should gain the route or the screen should be removed.
- Remove the CDN secret path (baseApi's x-cdn-secret header). Confirm that no production build sets VITE_CDN_TOKEN.
- Stop sending the Zoho owner key from the client. Resolve it on the server from the authenticated session.
- Remove the ZohoCrmTestConsole route from production builds.

Near term (within a quarter)

- Replace role and plan checks with server-issued permissions returned with the user, and enforce them in the backend. Remove the inert blockRoles, PaidOnly and 402 paths until a real plan model exists.
- Add a route-level code split for each top-level area, so the first load downloads only the shell and the session.
- Consolidate the 38 duplicated endpoint names. Delete the five dead API modules.
- Add a script for tests and lint to package.json, and run both in continuous integration alongside the build.
- Add a contract test that runs the reconciliation in this document on every change to either side, so route drift is caught before release.

Strategic

- Decide the real-time model (Section 11) and make it consistent across frontend and backend.
- Move session handling to httpOnly cookies, or accept and document the localStorage risk with a stated mitigation.
- Replace the two hand-written HTML filters with one vetted sanitiser at every render point.
- Store notification preferences on the server so that they are honoured wherever notifications are generated.

17 Limitations

- The reconciliation is static. It proves that a route with the same method and path exists, not that the handler works with the payload the screen sends.
- Dynamic URLs (seven calls) were not resolved. Their correctness needs a request-level test.
- Guard status is taken from the controller file, so it is conservative. The count of unguarded working calls is a lower bound.
- Behaviour was not exercised in a browser. Polling, notification delivery and session expiry were reviewed in source only.
- The frontend's unit tests were not run, because no script runs them. Lint was not run.
- The production build was verified, but the frontend was not run against a live backend.

Appendix A Endpoint mismatch register

Each row is a call from a live frontend module that does not match a backend route with the same method and path. The backend equivalent is a suggestion from the route inventory, not a confirmed replacement.

Category	Method	Frontend call	Module	Backend status or equivalent
Absent path	GET	/vendor-shortlists	vendors/vendor.api.js	project-shortlists / shortlist-entries (exist)
Absent path	POST	/vendor-shortlists	vendors/vendor.api.js	project-shortlists / shortlist-entries (exist)
Absent path	POST	/vendor-shortlists/\${shortlistId}/items	vendors/vendor.api.js	project-shortlists / shortlist-entries (exist)
Absent path	GET	/projects/\${id}/status-checklist	projects/project.api.js	gates/projects/:id/checklist (exists)
Absent path	GET	/boq-catalog	boq/boq.api.js	none found
Absent path	POST	/architect/visits	site-ops/site-ops.api.js	site-ops/visits (log and assignment routes exist; generic CRUD does not)
Absent path	POST	/architect/visits/generate	site-ops/site-ops.api.js	site-ops/visits (log and assignment routes exist; generic CRUD does not)
Absent path	GET	/architect/visits	site-ops/site-ops.api.js	site-ops/visits (log and assignment routes exist; generic CRUD does not)
Absent path	GET	/architect/visits/progress/\${projectId}	site-ops/site-ops.api.js	site-ops/visits (log and assignment routes exist; generic CRUD does not)
Absent path	GET	/architect/visits/\${id}	site-ops/site-ops.api.js	site-ops/visits (log and assignment routes exist; generic CRUD does not)
Absent path	PATCH	/architect/visits/\${id}	site-ops/site-ops.api.js	site-ops/visits (log and assignment routes exist; generic CRUD does not)
Absent path	DELETE	/architect/visits/\${id}	site-ops/site-ops.api.js	site-ops/visits (log and assignment routes exist; generic CRUD does not)
Absent path	POST	/delivery-challans/\${id}/complete	procurement/delivery-challan.api.js	none; only base, :id and :id/receive exist

Category	Method	Frontend call	Module	Backend status or equivalent
Absent path	PATCH	/delivery-challans/\${id}/status	procurement/delivery-challan.api.js	none; only base, :id and :id/receive exist
Absent path	GET	/delivery-challans/\${id}/history	procurement/delivery-challan.api.js	none; only base, :id and :id/receive exist
Absent path	GET	/delivery-challans/\${id}/items/\${itemId}/history	procurement/delivery-challan.api.js	none; only base, :id and :id/receive exist
Absent path	PATCH	/delivery-challans/\${id}/items/\${itemId}	procurement/delivery-challan.api.js	none; only base, :id and :id/receive exist
Absent path	POST	/delivery-challans/\${id}/items/\${itemId}/accept	procurement/delivery-challan.api.js	none; only base, :id and :id/receive exist
Absent path	POST	/delivery-challans/\${id}/items/\${itemId}/reject	procurement/delivery-challan.api.js	none; only base, :id and :id/receive exist
Absent path	POST	/delivery-challans/\${id}/partial-receive	procurement/delivery-challan.api.js	delivery-challans/:id/receive (exists)
Absent path	POST	/delivery-challans/\${id}/history	procurement/delivery-challan.api.js	none; only base, :id and :id/receive exist
Absent path	GET	/site-ops/dashboard	procurement/site-ops.api.js	site-ops/daily-reports/dashboard (exists)
Absent path	POST	/site-ops/qc	procurement/site-ops.api.js	site-ops/checklists (exists)
Absent path	GET	/site-ops/qc/\${id}	procurement/site-ops.api.js	site-ops/checklists (exists)
Absent path	GET	/site-ops/qc/projects/\${projectId}/history	procurement/site-ops.api.js	site-ops/checklists (exists)
Absent path	GET	/site-ops/qc/handoff-status	procurement/site-ops.api.js	site-ops/checklists (exists)
Absent path	GET	/site-ops/qc/projects/\${projectId}/handoff-status	procurement/site-ops.api.js	site-ops/checklists (exists)
Absent path	GET	/site-ops/qc/projects/\${projectId}/history	procurement/site-ops.api.js	site-ops/checklists (exists)
Absent path	GET	/procurement/site-inventory	procurement/procurent.api.js	inventory/stock/... (different path and shape)
Absent path	GET	/procurement/site-inventory/\${id}	procurement/procurent.api.js	inventory/stock/... (different path and shape)
Absent path	POST	/procurement/site-inventory/transactions	procurement/procurent.api.js	inventory/stock/... (different path and shape)
Absent path	GET	/procurement/site-inventory/reconcile/\${purchaseOrderId}	procurement/procurent.api.js	inventory/stock/... (different path and shape)
Absent path	POST	/inventory/receive	procurement/inventory.api.js	inventory/receive-delivery (exists)

Category	Method	Frontend call	Module	Backend status or equivalent
Absent path	POST	/inventory/opening-stock	procurement/inventory.api.js	none under this path
Absent path	POST	/inventory/reverse/\${id}	procurement/inventory.api.js	none under this path
Absent path	GET	/inventory/material-history/\${projectId}/\${materialId}	procurement/inventory.api.js	none under this path
Absent path	GET	/inventory/site-stock/\${projectId}/\${siteId}	procurement/inventory.api.js	none under this path
Absent path	GET	/documents/\${id}/reki	documents/document.api.js	none found
Absent path	GET	/projects/\${projectId}/documents-workspace	documents/document.api.js	none found
Method mismatch	PATCH	/material-master/\${id}	components/ PurchaseOrderForm.jsx	backend has GET
Method mismatch	POST	/boqs/\${boqId}/export/excel	boq/boq.api.js	backend has GET
Method mismatch	POST	/site-ops/snag-lists/\${id}/export	site-ops/snag-lists.api.js	backend has GET
Method mismatch	POST	/dpr/admin-documents/\${id}/export	site-ops/admin-dpr.api.js	backend has GET
Method mismatch	POST	/dpr/admin-reports/export	site-ops/admin-dpr.api.js	backend has DELETE/GET/PATCH
Method mismatch	POST	/projects/\${projectId}/planners/workbook.xlsx	documents/project-planner.api.js	backend has GET
Dynamic, unverified	GET	/auth/google/authorize\${scopes ?	auth/authConnectors.api.js	verify the composed URL at runtime
Dynamic, unverified	GET	/auth/microsoft/authorize-url\${scopes ?	auth/authConnectors.api.js	verify the composed URL at runtime
Dynamic, unverified	GET	/auth/zoho/authorize-url\${scopes ?	auth/authConnectors.api.js	verify the composed URL at runtime
Dynamic, unverified	POST	/site-ops/snag-lists\${id ?	site-ops/snag-lists.api.js	verify the composed URL at runtime
Dynamic, unverified	GET	/dpr/admin-\${kind}	site-ops/admin-dpr.api.js	verify the composed URL at runtime
Dynamic, unverified	POST	/dpr/admin-\${kind}/\${id ?	site-ops/admin-dpr.api.js	verify the composed URL at runtime
Dynamic, unverified	DELETE	/dpr/admin-\${kind}/\${id}	site-ops/admin-dpr.api.js	verify the composed URL at runtime

Table A.1. 52 entries: 39 absent path, 6 method mismatch, 7 dynamic and unverified.

Appendix B Reproduction

The results can be reproduced from the archives with these steps. The reconciliation scripts parse each builder block, pair the URL with its method, and match it against controller decorators.

```
cd rippotai-erp-frontend
npm ci --ignore-scripts
npx vite build                # passes; 4.27 MB main bundle

# Endpoint reconciliation (summary of method):
#   1. For each builder.query / builder.mutation block in src/api:
#       take the url (or query() return string) and the method (default GET for query, POST for
#       mutation)
#   2. For each @Get/@Post/@Put/@Patch/@Delete in backend/src controllers:
#       full path = controller prefix + method path
#   3. Match on method, segment count, and literal segments (params are wildcards)
#   4. Exclude modules that no other module imports

# Guard status: a controller file containing JwtAuthGuard or CdnGuard is treated as guarded
```